

CONTENTS

- Overview
- 1 Reactive generation versus proactive goal pursuit
- 2 What a generative model is actually doing
- 3 The agent loop: perceive, decide, act, learn
- 4 The shared substrate: an LLM as reasoning engine
- 5 Generative in practice: the human curates
- 6 Agentic in practice: multi-step work under a single goal
- 7 Chain-of-thought: the internal dialogue before the action
- 8 Systems that know when to generate and when to act
- Glossary
- Check yourself
- Where to go next

Generative vs Agentic AI: One Call versus a Loop

Understand what actually changes when a model stops answering and starts acting.

For engineers deciding whether a feature needs a model call or an agent · 26 July 2026

[Watch the source video](#)

[Read the condensed version](#)

[Read the highlights](#)

BEFORE YOU START

- How a chat completion API call works: messages in, message out
- Comfort reading Python
- Some exposure to function/tool calling, though the lesson explains it

Overview

The difference between generative and agentic AI is not a difference in model. It is a difference in control flow. A generative system is one request and one response: you prompt, it produces, it stops. An agentic system takes that same model and puts it inside a loop that observes, decides, acts on the world, and feeds the result back in.

That structural change is what produces every other difference. A single call is cheap, fast, bounded and easy to evaluate. A loop is none of those things — it can run for minutes, call external systems, cost an unpredictable amount, and fail in ways a single call cannot. In exchange it can pursue a goal you specified once, across many steps, without you supervising each one.

Both rest on the same foundation. A large language model provides the reasoning in both cases; in the agentic case it is doing something narrower and more specific than people usually assume — it chooses the next action and interprets the last result. Everything else in an agent is ordinary software.

KEY TAKEAWAYS

- ✓ Generative and agentic AI differ in control flow, not in the model — one is a call, the other is a loop containing that call.
- ✓ A generative system is reactive: it waits for a prompt, produces content, and stops.
- ✓ An agentic system is proactive: given a goal, it perceives, decides, acts, and learns from the result, repeating with minimal human input.
- ✓ The same LLM serves as the reasoning engine in both; in an agent it decides the next action and interprets the last observation.
- ✓ Chain-of-thought decomposition is how an agent turns a vague goal into an ordered sequence of executable steps.
- ✓ Generative workflows keep a human curating every output; agentic workflows spend human attention on the goal and the guard rails instead.
- ✓ A loop has no natural stopping point, so termination conditions, step budgets, and failure handling are your responsibility, not the model's.
- ✓ Cost and latency become unbounded and data-dependent the moment you move from one call to a loop.

01 Reactive generation versus proactive goal pursuit

0:00

One produces content and stops. The other pursues an objective across many actions. The gap between them is a while loop.

Generative systems — chatbots, image generators, code completion — are fundamentally reactive. They wait. Nothing happens until you prompt them, and once they have produced their output their work is over. They take no further step without your input, and they have no notion of an objective that outlives the current response.

Agentic systems are proactive. They usually still start from a prompt, but that prompt is treated as a goal to pursue rather than a request to answer. The system then takes a series of actions toward that goal, deciding for itself what to do next, and continues with minimal human intervention until it decides it is done.

Hold onto what has and has not changed. The model is the same model. The API is the same API. What changed is that somebody wrapped it in a loop, gave it the ability to call out to the world, and fed the results of those calls back into its context. That is the entire architectural distinction, and every practical consequence — cost, latency, failure modes, testability — follows from it.

KEY POINTS

- Generative systems are reactive: prompt in, content out, done.
- Agentic systems are proactive: goal in, sequence of actions out, done when the goal is met.
- The model is identical; the control flow around it is what differs.
- An agent's prompt is an objective that outlives any single response.
- Every practical difference follows from the loop, not from the model.

The same model, two shapes of program

This is the whole distinction in fourteen lines. The top is a function call. The bottom is a control structure that happens to contain one.

```
# generative: one call, one result, done
answer = model.generate(prompt)

# agentic: the call is now inside a loop that can act on the world
state = {"goal": goal, "history": []}

while not done(state):
    observation = perceive(state)          # look at the world
    action      = model.decide(state, observation)  # the LLM's job
    result      = execute(action)          # actually do it
    state       = update(state, action, result)    # learn from it
```

Where the human sits

Not a difference in how much human effort is involved, but in where it is spent. Generative work spends it per output. Agentic work spends it up front on the goal and the constraints — and then on reviewing what already happened.

```
generative : human → prompt → output → human reviews → prompt → output → ...
            └─ attention on every single result ─┘

agentic    : human → goal → [ agent loops, acts, retries ] → result → human reviews
            └─ attention on the goal and the guard rails ─┘
```

02 What a generative model is actually doing

0:40

Sophisticated pattern completion: statistical relationships learned from a large corpus, used to predict what plausibly comes next.

A generative model learned the statistical relationships in its training data — between words, between pixels, between audio samples — and at generation time it uses those relationships to predict what should come next given what has come so far. Text models produce a probability distribution over the vocabulary at each position and sample from it. That is the whole mechanism.

Calling it pattern matching is accurate but easy to under-read. The patterns are extremely rich, and the model is conditioning on the entire prompt at every step, which is why the output is coherent over long spans and responsive to detail. But note what is absent: there is no plan, no world model being consulted, no check against reality. A step is plausible or it is not, and the model was optimised for plausible.

Different media use different architectures — transformers for text and code, diffusion models for most images and audio — but the framing holds across them. The output is a sample from a learned distribution conditioned on your input.

One consequence matters more than the rest for what follows. Because the model produces the plausible continuation rather than the verified one, a generative system needs something outside it to check the result. In a generative workflow that something is a human. In an agentic workflow, it has to be built.

KEY POINTS

- Generation is next-token prediction: a distribution over the vocabulary, sampled at each position.
- Coherence comes from conditioning on the whole prompt at every step, not from a stored plan.
- Text and code use transformers; images and audio typically use diffusion models.
- The model optimises for plausible continuations, not verified ones.
- Something outside the model must check the output — a human in generative use, machinery in agentic use.

One step of generation

The model's actual output is the distribution, not the word. Sampling turns it into a token, and the token is appended and fed back. Temperature controls how much probability mass outside the top candidate gets a chance — which is also why the same prompt can produce different results.

```
prompt = "The capital of France is"

# the model produces a distribution over the whole vocabulary
logits = model(prompt)          # shape: [vocab_size]
probs = softmax(logits / temperature)

# " Paris"    0.94
# " located"  0.02
# " a"        0.01
# ...

next_token = sample(probs)      # append, then repeat with the longer prompt
```

Why plausible is not the same as correct

Both continuations are statistically excellent. Only one is true, and nothing in the generation mechanism distinguishes them — which is precisely the gap that human review, or an agent's verification step, exists to close.

```
prompt: "The library function `parse_config` accepts a parameter called"

" path"      - plausible, and true
" filename"  - equally plausible, and invented

# the model has no way to check; it was trained to continue, not to verify
```

03 The agent loop: perceive, decide, act, learn

1:22

Four stages, repeated. Everything that makes an agent an agent lives in this cycle rather than in the model.

An agentic system moves through a life cycle. It perceives its environment — reads files, queries an API, checks the current state of whatever it is working on. It decides an action to take. It executes that action. Then it learns from the output of that action, and the cycle begins again. Round and round until the goal is met.

Only the decide step is the model. Perceive is I/O. Execute is a function call. Learn, in most production agents, means appending the result to a message history so the next decide step can see it. That accounting is worth internalising, because it tells you where bugs will actually live: not in the model's reasoning, but in tool definitions, error handling, and state management.

What this structure buys is recovery. A generative call that produces something wrong produces something wrong and stops. An agent that takes a wrong action sees the result of that action — a failing test, a 404, an empty result set — and gets another turn with that information in context. Fallibility per step matters much less when the loop can observe and correct.

What it costs is control. The loop has no natural stopping point. Nothing in the model guarantees it will decide it is finished, or that it will not oscillate between two actions forever. Termination is a design problem you own: a step budget, a wall-clock timeout, a no-progress detector, and a rule for what happens when the budget runs out.

KEY POINTS

- The cycle is perceive → decide → act → learn, repeated until done.
- Only 'decide' is the model; the other three stages are ordinary software.
- Feeding action results back into context is what allows an agent to recover from a bad step.
- The loop has no inherent stopping condition — step budgets and timeouts are your responsibility.
- Most agent bugs live in tool definitions and error handling, not in model reasoning.

A working agent loop

This is what almost every agent framework wraps. Note the three defences that have nothing to do with AI: a hard step budget, tool errors fed back as observations rather than raised, and a definite outcome when the budget is exhausted.

```
def run_agent(goal, tools, max_steps=20):
    messages = [
        {"role": "system", "content": SYSTEM_PROMPT},
        {"role": "user", "content": goal},
    ]

    for step in range(max_steps):
        reply = model.chat(messages, tools=tools)    # decide

        if not reply.tool_calls:                    # the agent says it is done
            return reply.content

        messages.append(reply)

        for call in reply.tool_calls:               # act
            try:
                result = tools[call.name](**call.arguments)
            except Exception as e:
                result = f"ERROR: {e}"              # failures are observations,
                                                    # not crashes

            messages.append({                       # learn
                "role": "tool",
                "tool_call_id": call.id,
                "content": str(result),
            })

    raise StepBudgetExceeded(goal, messages)        # never loop forever
```


Tools are the agent's senses and hands

An agent can only perceive and act through the tools you give it. The description field is not documentation — it is the only thing the model uses to decide when to reach for this tool, so it is functionally part of your prompt.

```
{
  "name": "search_inventory",
  "description": "Check current stock and price for a product across
                  all warehouses. Use before promising availability
                  to a customer.",
  "parameters": {
    "type": "object",
    "properties": {
      "sku": {"type": "string", "description": "Product SKU, e.g. 'AB-1042'"},
      "region": {"type": "string", "enum": ["EU", "US", "APAC"]}
    },
    "required": ["sku"]
  }
}
```

Detecting a stuck loop

A step budget catches runaway agents eventually. A no-progress check catches them sooner and more cheaply, by noticing that the agent is repeating an action that already failed.

```
def is_stuck(history, window=3):
    recent = [(a.name, a.arguments) for a in history[-window:]]
    return len(recent) == window and len(set(map(str, recent))) == 1

# same tool, same arguments, three times running → intervene rather than
# burning the rest of the budget
```

04 The shared substrate: an LLM as reasoning engine

2:03

Both approaches usually rest on the same model. In an agent, its job is narrower than people assume: choose the next action, interpret the last result.

Large language models are the common foundation. They power chatbots directly, and they provide the reasoning that drives agentic systems. Other generative media reach for other architectures — diffusion models for images and audio — but the reasoning layer in agents is overwhelmingly an LLM.

It is worth being precise about what the model contributes inside an agent, because 'the LLM is the brain' oversells it. On each turn the model receives the goal, the history so far, and the schemas of the available tools. It emits either a tool call with arguments, or a final answer. That is a classification-and-argument-filling job under uncertainty, repeated. It is not planning in any strong sense, and it holds no state — the message history is the state, and you own it.

That framing pays off immediately in debugging. When an agent misbehaves, the useful question is rarely 'why did the model reason badly'. It is: what exactly was in the context at that turn? Was the tool description ambiguous? Did the previous tool return something unreadable? Was the relevant detail forty messages back and pushed toward the middle of a long context? Nearly every agent failure is legible if you dump the message list at the failing step.

KEY POINTS

- LLMs are the reasoning layer for agents and the generator for chatbots — the same component in two roles.
- Diffusion models cover most image and audio generation; the agentic reasoning layer is an LLM.
- Per turn, the model's job is to emit a tool call with arguments, or a final answer.
- The model is stateless; the message history is the state and the application owns it.
- Debug agents by inspecting the exact context at the failing turn, not by theorising about reasoning.

What the model sees on turn four

The model has no memory between turns. Everything it can use is in this list, which the application assembled. If a needed fact is not here, the model cannot use it — and that is a context construction bug, not a reasoning failure.

```
[
  {"role": "system",
   "content": "You are a shopping agent. Confirm price before purchase."},
  {"role": "user",   "content": "Buy the AB-1042 under 200 EUR"},

  {"role": "assistant", "tool_calls": [search_inventory(sku="AB-1042", region="EU")]},
  {"role": "tool",      "content": "in_stock: 3, price: 189.00 EUR, warehouse: NL"},

  {"role": "assistant", "tool_calls": [check_price_history(sku="AB-1042", days=30)]},
  {"role": "tool",      "content": "min: 175.00, max: 210.00, current: 189.00"},

  # turn 4: given all of the above, emit the next tool call or a final answer
]
```

A debugging habit worth adopting

Log the full context at every turn from the first day of the project. When an agent does something inexplicable in production, this transcript is the difference between a five-minute diagnosis and a week of guessing.

```
def decide(messages, tools, step):
    log.debug("step=%d context=%s", step, json.dumps(messages, indent=2))
    reply = model.chat(messages, tools=tools)
    log.debug("step=%d chose=%s", step, reply.tool_calls or "FINAL")
    return reply
```

05 Generative in practice: the human curates

3:28

Generative workflows produce possibilities. A person selects, corrects and directs — and that review step is load-bearing.

Consider a video creator using generative AI across a production: drafting and reviewing scripts, proposing thumbnail concepts, generating background music. At every one of those steps there is a human looking at what came back, judging whether it is what they wanted, and refining it when it is not — which it often is not.

The AI generates possibilities; the human curates them. That division is the point, not a limitation people are working to remove. It suits tasks where the cost of a bad output is low, where quality is subjective enough that only the person asking can judge it, and where iteration is cheap and fast.

The engineering implication is that the review step is part of the system. If you build a generative feature and quietly remove the human — auto-publishing generated copy, committing generated code unreviewed — you have not built an efficient generative system. You have built an agentic one without the guard rails an agentic system needs.

KEY POINTS

- Generative workflows pair machine output with human judgement on every result.
- They suit low-stakes, subjective, fast-iteration tasks.
- The human review step is a component of the system, not overhead to optimise away.
- Removing the reviewer without adding verification turns a generative system into an unguarded agentic one.
- Cost and latency are bounded and predictable: one call, one result.

Generate options, do not generate the answer

A small design choice with a large effect on generative features. Returning several candidates makes the human's job selection rather than correction, which is faster and produces better outcomes than asking for one output and editing it.

```
def thumbnail_concepts(title, n=5):
    return [
        model.generate(
            f"Propose a thumbnail concept for a video titled {title!r}. "
            f"One sentence. Be specific and visual.",
            temperature=0.9,          # high: we want spread, not consensus
        )
        for _ in range(n)
    ]

# the human picks one and refines it – selection beats correction
```

06 Agentic in practice: multi-step work under a single goal

4:10

Agents earn their complexity on tasks that need ongoing management across many steps, where waiting for a human between each one defeats the purpose.

Take a personal shopping agent. Given a product to buy, it hunts for availability across platforms, monitors price movement over time, handles checkout, and coordinates delivery — largely by itself, coming back to you only when it genuinely needs something.

Every one of those characteristics is what justifies the loop. The task is multi-step, and the later steps depend on what the earlier ones found. It runs over time rather than at one moment. It requires reacting to a changing world. Break that into individual prompts with a human between each and you have not simplified anything — you have made the human the loop.

But apply the reverse test before building one. If the task is a single step, an agent is a slower and less reliable function call. If every step needs approval anyway, the agent is a worse interface than a form. And if the actions are irreversible — purchases, deletions, sent messages — you need the loop to be interruptible and every consequential action to be either reversible or explicitly confirmed. The agent that can spend money is exactly the one that needs a spending limit enforced outside the model, not requested inside the prompt.

KEY POINTS

- Agents fit multi-step tasks where later steps depend on earlier results.
- They fit work that unfolds over time and reacts to a changing environment.
- If a task is one step, an agent is a slow, unreliable function call.
- If every step needs approval, a form is a better interface than an agent.
- Irreversible actions need limits enforced in code, never merely requested in the prompt.

Enforce limits outside the model

The prompt says 'under 200 EUR'. That is a request. This is a guarantee. Any action with real consequences should pass through a check the model cannot talk its way past, because the model is a probabilistic system and your budget is not.

```
def purchase(sku, price_eur, *, budget_eur, confirmed):  
    if price_eur > budget_eur:  
        return f"REFUSED: {price_eur} exceeds budget {budget_eur}"  
    if not confirmed:  
        raise NeedsHumanApproval(sku=sku, price=price_eur)  
    return checkout(sku)  
  
# the model can request a purchase; only this function can make one
```

Deciding whether you need an agent at all

Four questions, and three of them have to point the same way. Most features that get built as agents fail the first or second and would have been better as a single call plus a form.

Does the task need more than one step?	no → single call
Do later steps depend on what earlier steps found?	no → a pipeline
Would a human have to wait around between steps?	no → keep the human
Are the actions reversible, or cheaply confirmable?	no → add approval
all four favourable → an agent is genuinely the right shape	

07 Chain-of-thought: the internal dialogue before the action

5:38

The agent breaks a complex goal into smaller logical steps by generating its own

| reasoning, then acts on the result.

Chain-of-thought reasoning is a process where the agent decomposes a complex task into smaller logical steps, in much the way a person tackles a hard problem — by talking it through before doing anything.

Give an agent 'organise a conference' and its internal dialogue goes roughly: first I need the requirements — size, duration, budget. Then I should research venues matching those parameters. Then, for the venues that qualify, I need to check availability. And so on. The agent is exploring the problem space in text before touching the world, and that exploration is generated by the same next-token prediction described earlier. Generation is the cognitive engine driving the agent's decision making.

The engineering value is that this reasoning is text, which makes it inspectable and testable in a way that a hidden plan would not be. You can log it, diff it across runs, and assert on it. When an agent takes an absurd action, the reasoning that preceded it is usually where the mistake is visible.

Two cautions. The reasoning is a generated artefact, not a guaranteed account of why the action was chosen — plausible-sounding justification for a bad choice is entirely possible. And decomposition is not free: every reasoning token is latency and cost on every turn. Which is why the interesting design question is usually how much thinking to buy per step, not whether to allow it.

KEY POINTS

- Chain-of-thought decomposes a goal into ordered, executable steps.
- The reasoning is generated text, so it can be logged, inspected and asserted on.
- Generation is the cognitive engine: the same mechanism, aimed at planning instead of prose.
- Stated reasoning is not a guaranteed explanation — a bad action can arrive with a convincing rationale.
- Reasoning tokens cost latency and money on every turn; depth is a budget decision.

Decomposition producing a checkable plan

Asking for the plan as structured output rather than prose makes it a data structure you can validate before any action runs — checking that dependencies are acyclic, that no step calls an unavailable tool, and that the cost is within budget.

```
PLAN_SCHEMA = {
  "steps": [
    {"id": 1, "action": "gather_requirements",
     "needs": [], "produces": "size, duration, budget"},
    {"id": 2, "action": "search_venues",
     "needs": [1], "produces": "candidate venues"},
    {"id": 3, "action": "check_availability",
     "needs": [2], "produces": "available venues"},
    {"id": 4, "action": "request_quotes",
     "needs": [3], "produces": "pricing"}
  ]
}

# validate before executing anything
assert is_acyclic(plan)
assert all(s["action"] in TOOLS for s in plan["steps"])
```

The reason–act trace

Interleaving reasoning with actions gives you a readable execution log. When step 3 does something unexpected, the thought immediately above it usually contains the error — a wrong assumption stated plainly.

Thought: I need the budget before I can filter venues. It was not given.

Action: ask_user(question="What is the budget per attendee?")

Obs: "120 EUR"

Thought: Budget known. 200 attendees x 120 = 24,000 total. Search within that.

Action: search_venues(capacity=200, max_total_eur=24000, city="Barcelona")

Obs: [3 venues]

Thought: Three candidates. Availability next, cheapest first.

Action: check_availability(venue_id="v-88", dates=["2026-09-14"])

The strongest systems are neither purely generative nor purely agentic — they explore options through generation and commit to them through action.

The useful end state is not one approach beating the other. It is systems that understand when to explore possibilities through generation and when to commit to a course of action, and that switch between the two deliberately.

The pattern to look for is a division by reversibility. Generation is cheap and reversible, so use it liberally for anything you can throw away: drafts, candidate plans, hypotheses, options. Action is expensive and often irreversible, so gate it — with a check, a budget, a confirmation, or a test that has to pass. In practice a well-built agent generates many possibilities and acts on very few of them.

The same split shows up in how you evaluate the thing. Generative quality is judged on the output, usually by a person or a rubric. Agentic quality is judged on whether the goal was reached, at what cost, with what side effects, and how it behaved when a step failed. Systems that mix both need both evaluations, and teams routinely build only the first — which is how an agent ships with good-looking outputs and no idea what it does on the unhappy path.

KEY POINTS

- Mature systems generate to explore and act to commit, switching deliberately.
- Divide by reversibility: generate freely, act under a gate.
- Good agents generate many candidates and execute few.
- Generative evaluation judges the output; agentic evaluation judges goal completion, cost and side effects.
- Building only output-quality evaluation leaves the failure path untested.

Generate three, verify, act on one

The generative and agentic halves in a single function. Candidates are free, so produce several. Execution is not, so only a validated candidate reaches it — and if none validate, escalating beats acting on the least bad option.

```
def act_on_best(goal, state, k=3):
    candidates = [propose_action(goal, state) for _ in range(k)]    # generative

    for action in rank_by_confidence(candidates):
        ok, reason = validate(action, state)                        # schema, budget, permissions
        if ok:
            return execute(action)                                  # agentic
        log.info("rejected %s: %s", action, reason)

    raise NoValidAction(goal, candidates)                          # escalate, do not improvise
```

Two evaluations, not one

Most teams build the left column and ship. The right column is what tells you whether the system is safe to leave running, and it is only measurable if you have instrumented the loop from the start.

generative eval	agentic eval
is the output good?	was the goal reached?
rated by human or rubric	measured against a success check
one call, one score	steps taken, tokens spent, wall time
	side effects (writes, spend, messages)
	behaviour when a tool failed
	behaviour at the step budget

Glossary

Generative AI

A system that produces content in response to a prompt and then stops. Reactive by construction: one request, one result.

Agentic AI

A system that pursues a goal across multiple self-directed actions, observing results and adjusting, with minimal human intervention.

Agent loop

The repeating cycle of perceive, decide, act and learn that defines an agent. The model supplies the decide step; the rest is ordinary software.

Tool (function calling)

A function exposed to the model with a name, description and parameter schema. Tools are the only way an agent can observe or change anything outside its context.

Chain-of-thought reasoning

Generating intermediate reasoning steps before answering or acting, so a complex goal is broken into smaller logical pieces.

Next-token prediction

The mechanism behind text generation: produce a probability distribution over the vocabulary and sample the next token, repeatedly.

Diffusion model

The architecture behind most image and audio generation, which produces output by iteratively denoising random noise.

Step budget

A hard cap on how many iterations an agent may run. Necessary because nothing in the model guarantees the loop terminates.

Message history

The accumulated record of instructions, model replies and tool results. Since the model is stateless, this list is the agent's entire memory.

Human in the loop

A required approval point before a consequential action proceeds. In generative workflows it is continuous; in agentic ones it is placed at specific gates.

Reason–act trace

An execution log alternating stated reasoning with the action taken, used to make an agent's behaviour inspectable after the fact.

Temperature

A sampling parameter controlling how much probability mass outside the top candidate is used. Higher values give more varied output.

Check yourself

Answer first, then open each one.

Two systems use the identical model, the identical prompt and the identical API. One is generative and one is agentic. What is actually different?

The control flow around the call. The agentic system puts the model inside a loop with tools, feeding each action's result back into context so the next turn can use it. Nothing about the model changed — which is why 'is this agentic?' is a question about your code, not about which model you bought.

An agent takes a nonsensical action at step 7. Why is 'the model reasoned badly' usually the wrong first hypothesis?

Because the model only sees the context you assembled for that turn. Far more often the tool description was ambiguous, a previous tool returned something unparseable, or the relevant fact was crowded out of a long context. Dump the message list at step 7 and the cause is usually visible immediately.

Your prompt instructs the agent never to spend more than 200 EUR. Why is that insufficient, and what replaces it?

A prompt is a request to a probabilistic system, so compliance is likely but not guaranteed, and any instruction can be undermined by unusual input or a long context. The limit has to be a check inside the purchase function, which refuses regardless of what the model asked for. Enforce consequential constraints in code; use the prompt to explain them.

Why does an agent tolerate a wrong step much better than a single generative call tolerates a wrong answer?

The agent observes the consequence — a failing test, a 404, an empty result — and gets another turn with that observation in context, so it can correct. A generative call produces its output and stops, with no opportunity to notice anything is wrong. Per-step reliability matters less when the loop can see and recover.

A task is a single well-defined step that a human must approve anyway. Why is an agent the wrong architecture?

The loop buys recovery across dependent steps and autonomy between them, and neither applies here. You pay unpredictable latency, unpredictable cost and non-determinism for benefits the task cannot use. A single call behind a form is faster, cheaper and testable.

Why is generating several candidate actions and executing one a better pattern than generating one action and executing it?

Generation is cheap and reversible while execution is expensive and often not, so the asymmetry favours producing spare candidates and spending the validation effort before committing. It also gives you a fallback: if the top candidate fails validation, the next is already available without another round trip.

Where to go next

- Implement the twenty-line agent loop above against two real tools and deliberately make one of them fail intermittently — then watch whether your error handling actually lets the agent recover or just loops.
- Take an agent you already have and log the full message list at every turn, then read the transcript of a run that went wrong; note how many failures are context construction rather than reasoning.
- Measure the cost distribution of an agentic feature across 100 runs: tokens and wall time per run. The spread, not the mean, is what breaks capacity planning.
- Build the no-progress detector and instrument how often it fires in normal operation — the number is usually higher than teams expect.
- Take a task you currently solve with an agent, rewrite it as a fixed pipeline with one model call per stage, and compare reliability, cost and latency. Many agentic features are pipelines that were never tested as pipelines.

- Read the ReAct paper (Yao et al., 2022) for the original formulation of interleaving reasoning traces with actions, which is the pattern nearly every agent framework now implements.
-

Lesson generated from a YouTube transcript with YouTube Lesson Builder.

Explanations are AI-generated. Check anything you intend to rely on.